Operating Systems and Parallel Computing

Lecture 5

Prof. Matteo Turilli

What Is `thr` In `htop`?

- `thr` = thread. What is a thread?
- Each process has an address space and a **single thread of control**.
- Often, it is useful to have multiple threads of control in the same address space, i.e., each thread is a ‘mini-process’ within a process with which it shares its memory.
- Why is it useful?
  a. Multiple possibly blocking activities in many applications. Simpler programming model when decomposing the application into multiple sequential threads that run in quasi-parallel (on 1 CPU). **Question**: where did we see the same pattern and advantages?

What Is `thr` In `htop`?

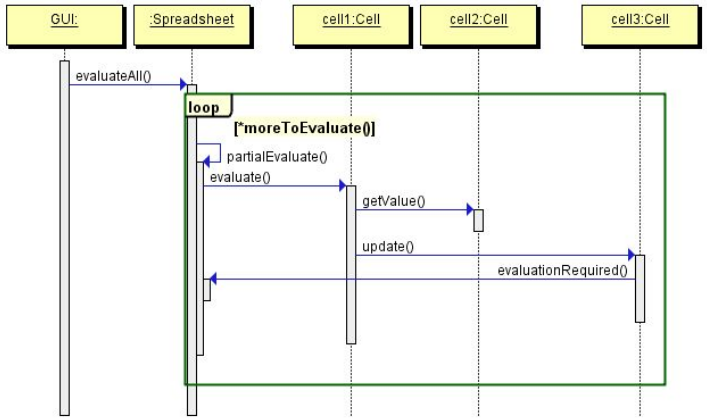
- `thr` = thread. What is a thread?
- Each process has an address space and a **single thread of control**.
- Often, it is useful to have multiple threads of control in the same address space, i.e., each thread is a ‘mini-process’ within a process with which it shares its memory.
- Why is it useful?
  a. Multiple possibly blocking activities in many applications. Simpler programming model when decomposing the application into multiple sequential threads that run in quasi-parallel (on 1 CPU). **Question**: where did we see the same pattern and advantages? **With the processes!**
  b. Ability for the parallel entities to share an address space and all of its data among themselves.
  c. 10–100 times faster than creating a process, useful when we have a lot of threads to create/destroy
  d. Performance: when all of threads are CPU bound they yield no performance gain, but when there is computing and I/O, threads allows these activities to overlap with a performance gain. **Why?**

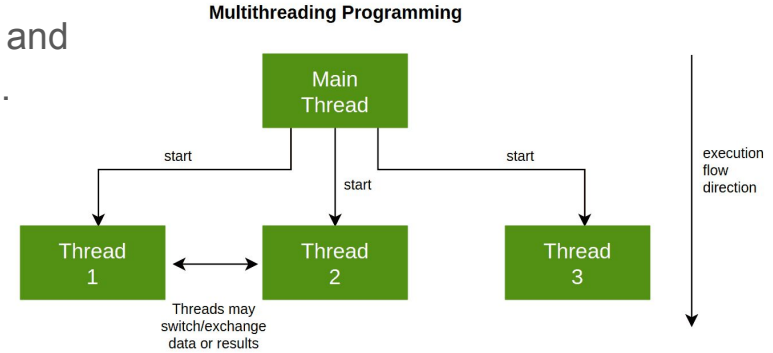
What Is `thr` In `htop`?

- `thr` = thread. What is a thread?
- Each process has an address space and a **single thread of control**.
- Often, it is useful to have multiple threads of control in the same address space, i.e., each thread is a ‘mini-process’ within a process with which it shares its memory.
- Why is it useful?
  a. Multiple possibly blocking activities in many applications. Simpler programming model when decomposing the application into multiple sequential threads that run in quasi-parallel (on 1 CPU). **Question**: where did we see the same pattern and advantages? **With the processes!**
  b. Ability for the parallel entities to share an address space and all of its data among themselves.
  c. 10–100 times faster than creating a process, useful when we have a lot of threads to create/destroy
  d. Performance: when all of threads are CPU bound they yield no performance gain, but when there is computing and I/O, threads allows these activities to overlap with a performance gain. **Why? Because I/O blocking threads allows to run computing threads while waiting, like with processes.**
  e. Performance: execute in parallel on systems with multiple CPUs/cores.

Threads: Examples

- Spreadsheet that computes the predicted annual profit of a company
- Hundreds of pages + thousands of formulas based on hundreds of input variables.
- Without threads:
  a. Updating cells is blocking
  b. Saving the spreadsheet on disk is blocking
  c. While computing cells or saving on disk, user input has to wait the operation finishes. Keyboard and mouse freeze, and inputting new values ‘does not work’ for a fraction of time.
- With three threads:
  a. Background thread: does the recomputation
  b. Interactive thread: allows the user to make additional changes while the computation is going on
  c. Save thread: handles periodic backups to disk on its own.





No Such a Thing As a Free Lunch

- Threads share memory. That gives them speed and power… And danger.
- **Race condition**:
  a. **Definition**: two threads read the same value, each modifies it, and the result depends on which thread happened to modify the value first.
  b. **Classic example**: two workers each add 1 to a shared counter. Worker A and B reads 10, A adds 1, saves 11, B adds 1, saves 11. The counter ends up at 11, not 12.
  c. **Silent killer**: The answer is wrong, but no exception is risen. The program silently makes mistakes that are difficult and time consuming to debug, usually also for a LLM.
  d. **Non-deterministic**: the same code on the same data can give a different answer on different runs,
  e. **Timing-dependent**: notoriously, they disappear when you add a print statement to investigate them, e.g., B arrives after A writes due to time taken to print and the counter is 12.
- Processes don't have this problem by default: they don't share memory, so they can't race on it.
- Processes have the **mirror-image problem**: to cooperate, they must copy data to each other which costs time and requires safe coordination with 3+ processes.

No Such a Thing As a Free Lunch

- **Mutual exclusion rule**: code that access shared data is called a “critical region” and we must have at most one thread inside the critical region at a time.
- Solutions to enforcing this rule:
  a. **Sleep awaken**: whoever leaves the region awakens the next thread.
  b. **Mutex** ("mutual exclusion lock"): lock before entering the critical region, unlock after.
  c. **Semaphore**: a counter for when we must have "at most N at a time" rather than "exactly one".
  d. **Built-in**: build the locking into the language, so we cannot forget to unlock, e.g., Python with lock.
  e. **Message passing:** not sharing memory at all, send copies instead as with processes. This is what multiprocessing queues and MPI do, and it's the model of Part II.
  f. **Barrier**: wait until every thread reached a condition, then all threads start again.
- Every one of these costs time. E.g., a lock serializes execution because the protected part runs one-at-a-time no matter how many cores we have.
- Priority inversion: a low-priority thread with a lock can stall a high-priority one.
- Avoiding locks altogether, e.g., techniques like read-copy-update (RCU), Multi-Version Concurrency Control (MVCC), Software Transactional Memory (STM)

Why User and Kernel Modes?

- **Goal**: share one machine safely without trusting every application.
- Protecting the system from potential failures or malicious activities by managing how applications interact with system resources like files, memory, I/O, and network.
- **User Mode**:
  a. **Isolation**: Applications run in user mode within their own virtual memory space. This means one application cannot access or modify the memory of another application.
  b. **Crash Protection**: If an application crashes, it does not affect the operating system or other applications, maintaining overall system stability.
  c. **Limited Access**: User mode restricts access to critical operating system resources, preventing applications from inadvertently damaging essential system data.
- **Kernel Mode**:
  a. **Shared Memory Space**: All code running in kernel mode shares a single virtual address space. This means that if a kernel-mode driver fails, it can lead to a system-wide crash.
  b. **Critical Operations**: Kernel mode handles essential tasks such as memory management (Lecture 6–7), process scheduling (lecture 10), and hardware communication.

Why Do We Need a Boundary?

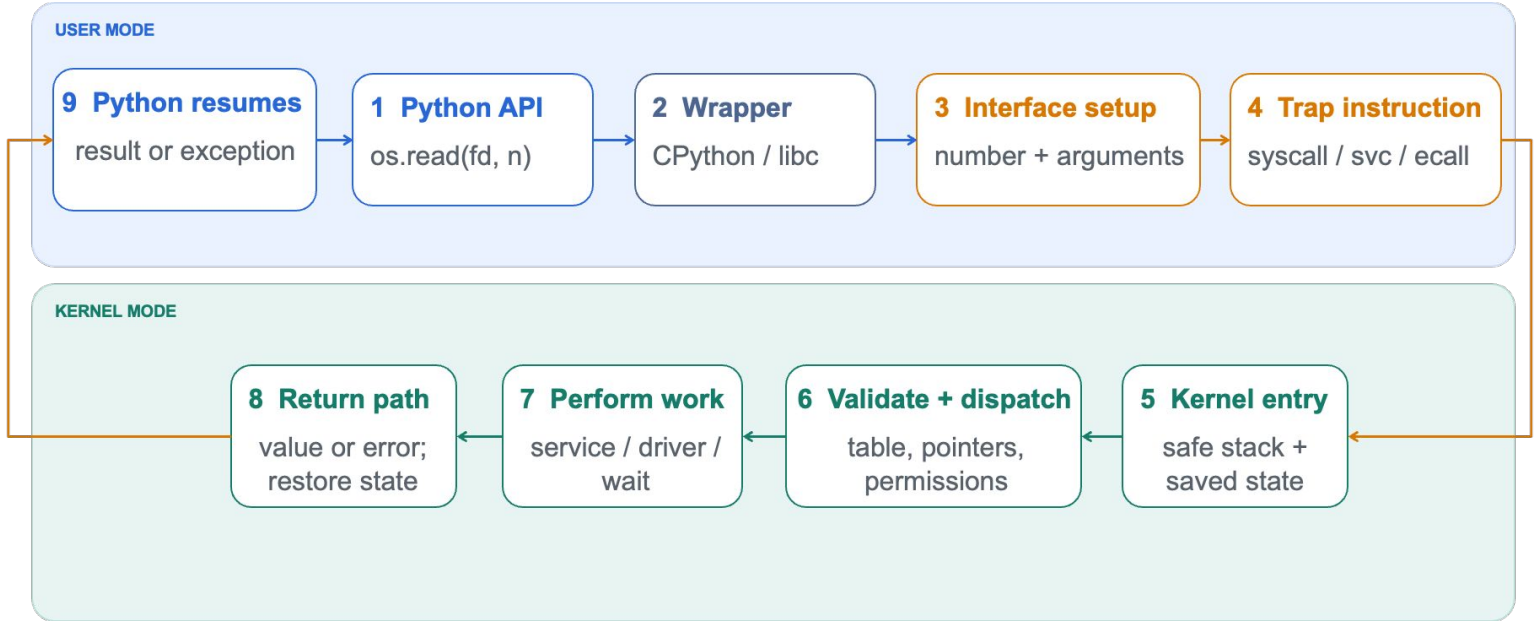
- Imagine every program could write to the disk directly, with no permission check.
  a. Any program could overwrite any file, including other users' files and the OS itself.
  b. A buggy loop could scribble over the memory of your browser, your bank session, everything.
  c. Nothing could be shared safely: no multi-user machines, no cluster, no cloud.
- Mechanisms to enable separation of user/kernel modes:
  a. **Applications make requests**: they name a service, supply arguments, and receive a value or error.
  b. **The OS kernel configures the rules**: it defines valid entry points, process memory mappings, identities, and permissions.
  c. **Hardware enforces the boundary**: the CPU and memory-management hardware check what the current code is allowed to do:
     - The processor records an architectural privilege level
     - Checks on memory permissions and privileged-instruction prevent unauthorized access.
- In this way, software (OS) and hardware (processing units) work together to maintain separation of concerns and thus the security and stability of the whole system.
- Software defines policy while hardware ensures that applications cannot simply bypass it.

Kernel Events and the System-call Trap

- Three types of events can enter the OS kernel:
  a. **System call:** an intentional request made by a running process. E.g., a process calling `read(fd, …)` asks the kernel to obtain data from a file.
  b. **Exception or fault**: an event detected by the CPU while executing the current instruction. E.g., a page fault, divide-by-zero operation, or invalid instruction.
  c. **Interrupt**: an event from outside the current instruction stream. E.g., a timer tick, completed disk operation, or arriving network packet.
- System call:
  a. **Setup**: placing the system-call number and arguments in the CPU registers.
  b. **Execute a special instruction**: requests to entry in protected kernel mode (called **trap**).
  c. **Hardware changes execution state**: the CPU changes privilege and executes the requested operation

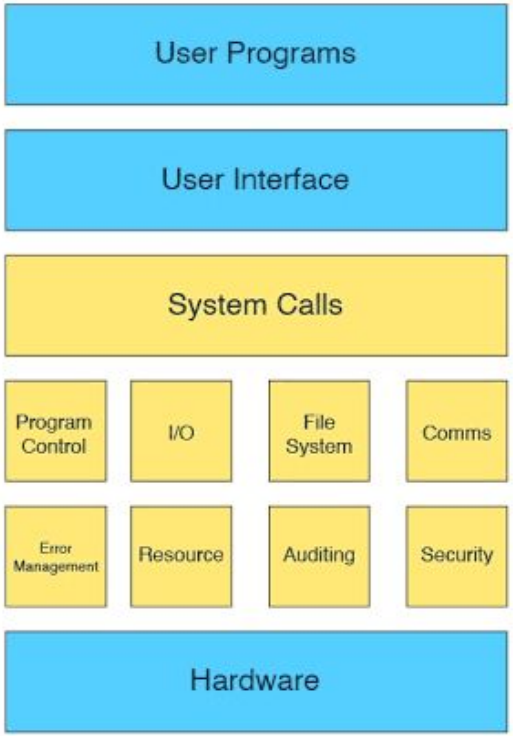
End to End: How Python Gets Kernel Work Done

- **Python API**: application code calls `pathlib`, `open()`, `os.read()`, `socket.recv()`, etc.
- **CPython runtime**: wrapper code converts Python objects and manages buffers and services.
- **System-call setup and trap**: low-level code executes the special entry instruction.
- **Kernel service**: the operating system validates and performs work involving files, processes, memory, networks, or devices.



Main Categories of System Calls

- Linux has roughly 300 system calls.
- We focus on Python (not C) and its syscall abstractions
- **Files**: `open`, `read`, `write`, `close`.
  - We will showcase and use these today.
- **Processes**: `fork`, `exec`, `wait`, `kill`.
  - How Session 4's parents/children are made.
- **Memory**: `brk`, `mmap`.
  - How to interface to RAM (Session 6).
- **Network**: `socket`, `connect`, `send`.
  - Network programming, required for every download, remote API calls (e.g., CGPT), etc.
- **Devices**: `ioctl` and friends.
  - Hardware programming, e.g., low-level robotics programming, microcontrollers, Linux drivers, etc.



Main Syscalls Interfaces in Python

- Files:
  - Open: `os.open(path, flags, mode)`
  - Read: `os.read(fd, n)`
  - Write: `os.write(fd, data)`
  - Close: `os.close(fd)`
- Processes:
  - Fork: `os.fork()`
  - Exec: `os.execve()`, `os.execv()`, `os.execl()`, …
  - Wait: `os.wait()`, `os.waitpid()`
  - Kill: `os.kill(pid, signal)`
- Network:
  - Socket: `socket.socket(...)`
  - Connect: `socket.connect(...)`
  - Send: `socket.send(...)`, `socket.sendall(...)`, `socket.sendto(...)`
- Devices:
  - Ioctl: `fcntl.ioctl(fd, request, arg)`
- Memory:
  - Mmap: `mmap.mmap(...)`

Syscalls And Python

- Python often provides both low-level and high-level interfaces:
  - For files, `os.open()`, `os.read()`, `os.write()`, and `os.close()` closely resemble Unix system calls. The built-in `open()` and file objects provide a safer and more convenient abstraction.
- We use low-level interfaces when precise control matters:
  - Low-level calls are useful in operating-system tools, servers, runtimes, databases, and embedded software.
- We normally use abstractions for application development as higher-level APIs reduce code and prevent common mistakes. E.g.:
  - With `open(...)` over manually calling `os.open()` and `os.close()`
  - `subprocess.run()` over manually combining `fork()`, `exec()`, and `wait()`
  - `Requests` or `httpx` over manually using sockets
  - Device-specific libraries over issuing raw `ioctl()` operations

Syscalls And Python

- File system calls reveal that files are OS-managed resources.
  - Open a file descriptor (fd) from the OS; read/write transfer bytes; close releases fd.
  - Python file objects hide this lifecycle, but underneath they rely on the same OS services.
- Process system calls explain how programs are created and controlled.
  - On Unix, fork creates a child process, exec replaces a process with another program, wait collects a child's result, and kill sends a signal.
  - Python exposes these through os, although subprocess and multiprocessing are usually more portable and easier to use.
- Memory calls explain how Python obtains memory from the OS.
  - Operating systems provide mechanisms such as brk and mmap to modify a process's address space.
  - Python's interpreter and memory allocator normally manage this automatically.
  - We use mmap.mmap() directly for memory-mapped files, shared memory, or efficient access to large data; Python does not provide a standard direct brk() wrapper.

Syscalls And Python

- Network abstractions are ultimately built on sockets:
  - socket, connect, send, and related calls establish endpoints and transfer bytes.
  - Libraries for HTTP, APIs, databases, and distributed systems sit on top of sockets and add protocols, encryption, authentication, timeouts, retries, and data serialization.
- Device calls provide a path to hardware-specific operations:
  - ioctl performs operations that do not fit normal byte-oriented read and write behavior.
  - In Python, it is exposed through fcntl.ioctl() on Unix.
  - Used for terminals, cameras, network interfaces, sensors, or custom drivers, but specialized libraries are usually safer and easier.
- We study system calls even when frameworks usually hide them:
  - They explain what high-level operations actually cost and why programs block, fail, leak resources, or behave differently across operating systems.
  - Understanding them helps us diagnose performance problems, choose suitable abstractions, reason about concurrency and security, and recognize when a framework cannot provide the control we need.

This is an interactive class

Start our course container now, the rest of this session is hands-on:

```
docker compose run --rm ospc
```

`strace`: Watching the “Conversation” With The OS

- strace runs a program and prints every system call it makes, as it makes them.
- It is a window onto the “conversation” between a program and the kernel.
- Let’s start small: what does a simple command actually ask for?

```
analyst@ospc:~$ strace ls data/sales_2025_sample.csv
execve("/usr/bin/ls", ["ls", "data/sales_2025_sample.csv"], 0xffffce249588 /* 14 vars */) = 0
brk(NULL)                               = 0xb69483277000
mmap(NULL, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_ANONYMOUS, -1, 0) = 0xf37ec92c0000
faccessat(AT_FDCWD, "/etc/ld.so.preload", R_OK) = -1 ENOENT (No such file or directory)
openat(AT_FDCWD, "/etc/ld.so.cache", O_RDONLY|O_CLOEXEC) = 3
fstat(3, {st_mode=S_IFREG|0644, st_size=8055, ...}) = 0
mmap(NULL, 8055, PROT_READ, MAP_PRIVATE, 3, 0) = 0xf37ec92be000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libselinux.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=264608, ...}) = 0
mmap(NULL, 337200, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) = 0xf37ec9234000
mmap(0xf37ec9240000, 271664, PROT_READ|PROT_EXEC, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf37ec9240000
munmap(0xf37ec9234000, 49152)           = 0
munmap(0xf37ec9283000, 13616)           = 0
mprotect(0xf37ec9270000, 61440, PROT_NONE) = 0
mmap(0xf37ec927f000, 8192, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0x3f000) = 0xf37ec927f000
mmap(0xf37ec9281000, 5424, PROT_READ|PROT_WRITE, MAP_PRIVATE|MAP_FIXED|MAP_ANONYMOUS, -1, 0) = 0xf37ec9281000
mmap(0xf37ec9240000, 196608, PROT_READ|PROT_EXEC|0x10, MAP_PRIVATE|MAP_FIXED|MAP_DENYWRITE, 3, 0) = 0xf37ec9240000
close(3)                                = 0
openat(AT_FDCWD, "/usr/lib/aarch64-linux-gnu/libgcc_s.so.1", O_RDONLY|O_CLOEXEC) = 3
read(3, "\177ELF\2\1\1\0\0\0\0\0\0\0\0\0\3\0\267\0\1\0\0\0\0\0\0\0\0\0\0\0"..., 832) = 832
fstat(3, {st_mode=S_IFREG|0644, st_size=133776, ...}) = 0
mmap(NULL, 263072, PROT_NONE, MAP_PRIVATE|MAP_ANONYMOUS|MAP_DENYWRITE, -1, 0) = 0xf37ec91ff000
```

`strace`: Watching the “Conversation” With The OS

- Even `ls` makes 100+ calls: loading libraries, checking files/dir, etc.
- The full trace is too long to read. `strace -c` just counts and summarises syscalls.
- This is the practical form: which calls, how many, how much time.

```
analyst@ospc:~$ strace -c ls data/sales_2025_sample.csv
data/sales_2025_sample.csv
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ----------------
 18.96    0.000080           6        13           read
 17.30    0.000073           7        10         1 openat
 13.03    0.000055          27         2         2 statfs
  7.11    0.000030           2        13           mprotect
  5.21    0.000022           2         9           rt_sigaction
  4.74    0.000020           3         6           ioctl
  4.50    0.000019           3         6           statx
  4.27    0.000018           2         9           close
  3.08    0.000013           4         3           brk
  3.08    0.000013           1        11           munmap
  2.61    0.000011           3         3           fcntl
  2.61    0.000011           5         2         2 faccessat
  2.37    0.000010           1         8           fstat
  2.13    0.000009           4         2           prlimit64
  1.90    0.000008           8         1           getrandom
  1.66    0.000007           2         3           sigaltstack
  1.42    0.000006           6         1           write
  1.18    0.000005           5         1           ppoll
  0.95    0.000004           4         1           sched_getaffinity
  0.71    0.000003           3         1           prctl
  0.71    0.000003           0        27           mmap
  0.47    0.000002           2         1           gettid
  0.00    0.000000           0         1           set_tid_address
  0.00    0.000000           0         1           set_robust_list
  0.00    0.000000           0         1           execve
  0.00    0.000000           0         1           rseq
------ ----------- ----------- --------- --------- ----------------
100.00    0.000422           3       137         5 total
```

`strace`: Watching the “Conversation” With The OS

- how many syscalls does Pandas to load a csv file?

```
analyst@ospc:~$ strace -c -f python3 -c "import pandas as pd; pd.read_csv('data/sales_2025_sample.csv')"
strace: Process 29 attached
% time     seconds  usecs/call     calls    errors syscall
------ ----------- ----------- --------- --------- ------------------
 70.44    0.042974         842        51           futex
 11.14    0.006797           6      1097           read
  5.76    0.003515           6       568           mmap
```

```
  0.02    0.000011          11         1           set_tid_address
  0.02    0.000011           5         2           gettid
  0.00    0.000001           1         1           sysinfo
  0.00    0.000000           0         1           epoll_create1
  0.00    0.000000           0         8         2 readlinkat
  0.00    0.000000           0         3           uname
  0.00    0.000000           0         1           getpid
  0.00    0.000000           0         1           execve
------ ----------- ----------- --------- --------- ------------------
100.00    0.061010           8      7420       368 total
```

How Much a Syscall Cost?

- A system call does not just ‘call a function’. It stops our program, switches privilege level, enters the kernel, does work, then switches all the way back.
- Typical cost of that round trip? on the order of a microsecond, i.e., a millionth of a second!
- Question: how bad can it be? What if a program makes a few million of them?

- Download `lecture_5_notebook.ipynb` from blackboard in Supporting Material
- Copy it into your `work` directory on your laptop (wherever you created it)
- Launch JupyterLab from our container: `docker compose up jupyter`

Exercises and Homework

- In the container:
  - Run the notebook `05_syscall_cost.ipynb`. What is the slowdown between 1-byte and 64-KB reads on **your** machine?
- `strace -c ls`:
  - Which system call is made most often? Can you guess why?
- `strace -c wc -l sales_2025_sample.csv`:
  - how many read() calls? What buffer size does that imply?
- Before next session: Crossing the kernel boundary costs time, per call. Execute and study the notebook and understand why/how we should batch our work.
- Next time: memory! Where your DataFrame actually lives, and why every process thinks it owns the machine.